Teacher guide: Lesson 1, My First Traffic Light

For: learners aged 10–15 (or adult beginners) who have done Pin Detective and the two Circuit Labs (if not, see the 5–10 min row). **New idea:** three LEDs taking turns, run by a `while True:` loop. **New part:** none (two more LED + resistor pairs). **Time:** about 35 minutes. **Works printed and offline** once `main.py` is on the board.

> This is a low-voltage circuit (3.3 V from the ESP32, powered by USB). Still, keep the USB cable **unplugged while wiring**, and have an adult check every circuit before it is plugged in. The lesson's 3D animation is a learning picture, not a safety test.

1. Before class (10 minutes, once per board)

1. The board needs MicroPython already on it (see `firmware/micropython/` in the course files).
2. Copy `lessons/01-traffic-light/main.py` onto each board as **`main.py`**, for example with Thonny (open the file, *File → Save as... → MicroPython device*, name it `main.py`). Thonny works offline once installed.
3. Unplug and plug the board back in. With nothing wired up yet, only the board's own small power light comes on. That is fine.
4. Use a **data** USB cable. Some cables only charge and cannot load programs.

`main.py` uses the lesson's default green time of **5 s**. The cycle is red 5 s → green 5 s → yellow 2 s, repeating (12 s).

2. Timing (35 minutes)

Minutes	What happens
0–5	Recap: "Last time you lit one LED." (Skip this if the class has not done the Circuit Labs.) Show a finished traffic light, or the lesson page. Hand out the worksheet; learners do **1. Predict**.
5–10	Parts check, LED legs (long leg = anode +, flat edge = cathode −) and the breadboard: the 5 holes in a row are joined underneath, and the middle gap splits each row in two. Water-pipe picture: pin = pump, wire = pipe, resistor = narrow pipe, LED = one-way glowing water wheel.
10–22	Build red, then yellow, then green, then the ground wires (table below). Learners tick each row on the worksheet.
22–25	Adult wiring check (section 5), then plug in USB.
25–30	**3. Observe**: count seconds, write the order. Try the demo: pull out only the long black wire from the − rail to the GND pin (all lights go dark), then plug it back.
30–35	**4. Explain** and discussion questions (section 7). Fast finishers: the "Make it yours" sum on the worksheet.

3. Parts list (per board or pair of learners)

Part	How many	Notes
ESP32 DevKit V1 (30 pins) with MicroPython and `main.py`	1	pins pointing up, not pushed into the breadboard
Breadboard with a blue − rail	1	half-size (400 holes) is enough
LEDs: red, yellow, green (5 mm)	1 each	new, untrimmed LEDs make the long leg easy to see
330 Ω resistors	3	colour bands orange, orange, brown (then gold); on 5-band resistors: orange, orange, black, black (then brown)
Female-to-male jumper wires	4	one each for D25, D26, D27 (red, yellow, green if you have them, to match the lights); one black for GND
Short black wires (male-to-male)	3	LED short-leg rows to the − rail
USB data cable (micro-USB or USB-C, to match the board)	1	and a computer or USB power bank

4. Wiring table (same as the lesson page)

Rows are the numbers along the breadboard; holes a–e are the top half, f–j the bottom half. The middle gap splits each row in two.

Light	ESP32 pin → hole	330 Ω resistor	LED long leg (+, anode)	LED short leg (−, cathode)	Black wire
Red	D25 → b7	d7 → g7 (across the gap)	i7	i8	j8 → − rail
Yellow	D26 → b15	d15 → g15	i15	i16	j16 → − rail
Green	D27 → b23	d23 → g23	i23	i24	j24 → − rail
Ground	GND pin next to D13 (same side as D25–D27) → − rail				

Why it is built this way, in one line each: each light has its own pin so the program can switch it alone; the resistor crosses the gap so electricity must pass through it; the short leg is in the next row so the LED is not skipped (a short circuit); the rail lets three LEDs share one GND wire. `Pin(25)` in the code means GPIO 25, printed **D25** on the board, not the 25th pin along the edge.

5. Step-by-step wiring check (USB unplugged)

Do this with the learner, pointing with a pencil. Tick each step for each light (red, then yellow, then green).

1. **Pin.** The wire starts on the pin printed D25 (D26, D27). Read the label; do not count pins.
2. **Top half.** That wire lands in row 7 (15, 23), any hole a–e. Nothing else is in that half-row except the resistor's top leg.
3. **Resistor.** One leg in the top half (d), one leg in the bottom half (g) of the **same row**. Bands orange-orange-brown.
4. **Long leg.** The LED's long leg is in the bottom half of that same row (i7, i15, i23).
5. **Short leg.** The short leg is in the **next** row (i8, i16, i24), not the same row as the long leg.

6. **Back to ground.** A black wire goes from that next row (j8, j16, j24) into the **blue − rail**, not the red + rail.
7. **Rail to GND.** One black wire joins the − rail to a GND pin. If the rail's blue line has a break in the middle (some long breadboards), every black wire must be on the same side of the break as the GND wire.
8. **No strays.** No bare legs touching each other; no part left in a row with nothing else in it.

Then plug in USB. Red should light first, within a second or two.

6. The five most likely faults (and fixes)

These come from how this circuit is built, not from classroom records; check them in this order.

What you see	Most likely cause	Fix
One light never comes on	LED in backwards	Turn the LED round: long leg in the resistor's row (i7/i15/i23), short leg in the next row.
One light never comes on, LED the right way	Wrong holes: both LED legs in one row (short circuit), or the resistor not crossing the gap	Redo that light with check steps 3–5.
No light ever comes on	Ground path broken: no wire from the − rail to GND, black wires on the + rail, or a split rail	Check steps 6–7. Once the loop is complete the lights come back on their own within one cycle (12 s); pressing EN to restart is optional.
Lights come on in the wrong order, or a light is dark	A wire is on the wrong ESP32 pin (pins counted, not read)	Read the printed labels: red D25, yellow D26, green D27.
Nothing at all, even the wiring is right; or it worked once and stopped	The program is not running: `main.py` not saved on the board (or saved under another name), a charge-only cable, or Thonny still connected and stopping it	Re-save as `main.py` on the device, close Thonny, unplug and replug. Try another cable.

Also: a light that is on but very dim usually has the wrong resistor (for example 10 kΩ: brown, black, orange). Swap it for 330 Ω.

7. Discussion questions

1. Why does every LED have **its own** resistor? (It limits the current for that LED, like a narrow pipe.)
2. What happened when the GND wire came out? Why? (No complete loop, so no current, so no light.)
3. Why is the short leg in a different row from the long leg? (Joined holes would let electricity skip the LED.)
4. The program sets all three lights in every phase. What could go wrong on a real road if two lights were on at once?
5. Why is yellow shorter than green? What should a driver do when the light turns yellow?
6. Where have you seen traffic lights near your school? Do you think a computer like this runs them?
7. If you wanted green to last 8 seconds, which number in the program would you change?

8. The program (for reference; the same as the lesson page with green set to 5 s, plus one comment)

```
# My First Traffic Light (ESP32)
from machine import Pin
import time

red = Pin(25, Pin.OUT)
yellow = Pin(26, Pin.OUT)
green = Pin(27, Pin.OUT)

while True:
    # Red light: stop
    red.value(1)
    yellow.value(0)
    green.value(0)
    time.sleep(5)

    # Green light: go
    red.value(0)
    yellow.value(0)
    green.value(1)
    # Green time in seconds. Change the 5 to make green longer or shorter (2 to 10).
    time.sleep(5)

    # Yellow light: get ready to stop
    red.value(0)
    yellow.value(1)
    green.value(0)
    time.sleep(2)
```

`value(1)` turns a light on, `value(0)` turns it off, `time.sleep(5)` waits 5 seconds, and `while True:` repeats the indented lines forever.